SIMULACIÓN DE EFECTOS ESPECIALES. LABORATORIO 1

Problema 1. Enfoque Denso. Importaciones y constantes.

Problema 1. Enfoque Denso.

```python
# AUTORES: Ignacio Cañizares / Pedro Pérez / César Vidal

# -------------------- #
# --- IMPORTACIONES --- #
# -------------------- #

import time
import numpy as np
from jacobi import jacobi

# ERORES de memoria de numpy.
from numpy._core._exceptions import _ArrayMemoryError


# ----------------- #
# --- CONSTANTES --- #
# ----------------- #

# Posibles valores del tamaño de la matriz cuadrada.
N_VALORES = [10, 20, 100, 200, 1000, 2000, 10000, 20000, 100000]

# Tolerancia permitida.s
TOL = 1e-6

# Número maximo de iteraciones permitidas.
MAXIT = 10000

# Límite de memoria de 16GB para no pasarse. Sin esto las últimas pruebas requieren hasta 60GBs de memoria.
LIMITE_MEMORIA = 2**(10) * 16
```

caesar

Problema 1. Enfoque Denso. Función construir matriz.

Problema 1. Enfoque Denso.

```python
def construir_A(n):

    """
    Para construir esta matriz densa cíclica tridiagonal A de tamaño n x n, hemos seguido estos pasos:

    1. Rellenamos nuestra matriz A con ceros
    2. Usamos np.fill_diagonal para establecer la diagonal principal con el valor 4.
    3. Usamos la misma función pero en este caso para rellenar las diagonales adyacentes a la principal
    4. Añadimos las esquinas de forma manual.

    """

    A = np.zeros((n, n))

    # Diagonal principal.
    np.fill_diagonal(A, 4)

    # Rellenamos la subdiagona y la superdiagonal con valor -1.
    np.fill_diagonal(A[1:], -1)
    np.fill_diagonal(A[:, 1:], -1)

    # Rellenamos las esquinas con -1.
    A[0, n - 1] = -1
    A[n - 1, 0] = -1

    return A
```

caesar

```
--- PASO 1: Construcción de A y medición de tiempo/memoria
```

n	Tiempo (s)	Memoria A
10	0.000043	0.0008 MB
20	0.000019	0.0031 MB
100	0.000049	0.0763 MB
200	0.000077	0.3052 MB
1000	0.000879	7.6294 MB
2000	0.001776	30.5176 MB
10000	0.008796	762.9395 MB
20000	0.019836	2.9802 GB
100000	0.594809	74.5058 GB

$$\mathbf{A} = \begin{pmatrix} 4 & -1 & 0 & \cdots & 0 & -1 \\ -1 & 4 & -1 & \cdots & 0 & 0 \\ 0 & -1 & 4 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 4 & -1 \\ -1 & 0 & 0 & \cdots & -1 & 4 \end{pmatrix},$$

Problema 1. Enfoque Denso. Construcción de matriz.

Problema 1. Enfoque Denso.

```python
# --- PASO 1: Construcción de A y medición de tiempo/memoria para cada n.

print("\n\n--- PASO 1: Construcción de A y medición de tiempo/memoria\n")
print(f"{'n':>8} {'Tiempo (s)':>12} {'Memoria A ':>16}")
print("-" * 42)

matrices = [None] * len(N_VALORES)

# ITERAMOS TODOS LOS VALORES N. # Introducimos un contador para ir almacenando las matrices.
for i, n in enumerate(N_VALORES):

    # Estimamos el tamaño en memoria de la matriz. Cada valor ocupa un byte.
    memoria_estimada = (n**2) / (1024**2)

    # Indicamos el caso de estar pasandonos en memoria.
    if memoria_estimada > LIMITE_MEMORIA:
        print(
            f"{n:>8} {'—':>12} {memoria_estimada / 2**(10 ):>14.1f} GB (omitido: excede límite)"
        )
        continue

    # Iniciamos para cada matriz su contador de tiempo.
    t0 = time.perf_counter()
    A = construir_A(n)
    b = np.ones(n)
    t1 = time.perf_counter()

    # Guardamos la matriz en el array:
    matrices[i] = A

    # Medimos la memoria real una vez creada.
    memoria_real = A.nbytes / (1024**2)

    # En el caso de ocupar mas de 1GB cambiamos el formato de visualización.
    if(memoria_real < 2**(10)):
        print(f"{n:>8} {t1 - t0:>12.6f} {memoria_real:>14.4f} MB")

    else:
        print(f"{n:>8} {t1 - t0:>12.6f} {memoria_real / 2**10 :>14.4f} GB")
```

caesar

```
--- PASO 2: Resolución con Jacobi

       n   Tiempo (s)      Error máx    Iters
------------------------------------------
      10     0.000818       9.54e-07       19
      20     0.000183       9.54e-07       19
     100     0.000219       9.54e-07       19
     200     0.000348       9.54e-07       19
    1000     0.004606       9.54e-07       19
    2000     0.026054       9.54e-07       19
   10000     0.550733       9.54e-07       19
   20000     2.307695       9.54e-07       19
  100000   385.455227       9.54e-07       19
```

Problema 1. Enfoque Denso. Función método de Jacobi.

Problema 1. Enfoque Denso.

```python
def jacobi(A, b, x0, tol=1.0e-6, maxit=200, verbose=True):
    if verbose:
        print("k\txk\t\t\terror")
        print(f"0\t{x0}\t")

    xk = x0
    for k in range(1, maxit + 1):
        xkprev = xk.copy()
        xk = xkprev + (b - A @ xkprev) / np.diag(A)
        error = np.linalg.norm(xk - xkprev, np.inf)
        if verbose:
            print(
                f"{k}\t{np.array2string(xk, formatter={'float': lambda x: f'{x:.6f}'})}\t{error:e}"
            )
        if error < tol:
            break
    else:
        print(f"Número máximo de iteraciones {maxit} alcanzado.")
        xk = None

    return xk, k  # devolver también k
```

pedro

Problema 1. Enfoque Denso. Método de Jacobi.

Problema 1. Enfoque Denso.

```python
# --- PASO 2: Resolución mediante Jacobi.

print("\n\n--- PASO 2: Resolución con Jacobi\n")
print(f"{'n':>8} {'Tiempo (s)':>12} {'Error máx':>14} {'Iters':>8}")
print("-" * 42)

for i, n in enumerate(N_VALORES):

    if matrices[i] is not None:

        A = matrices[i]

        # Los términos independientes son siempre uno.
        b = np.ones(n)

        # Las soluciones se obtienen en base float32.
        x0 = np.zeros(n)

        t0 = time.perf_counter()
        x_jac, iters = jacobi(A, b, x0, tol=TOL, maxit=MAXIT, verbose=False)
        t1 = time.perf_counter()

        # Calculamos el error sabiendo que la solución exacta es 0.5 para todos los casos.
        x_exacta = np.full(n, 0.5)
        error = np.max(np.abs(x_jac - x_exacta))

        print(f"{n:>8} {t1 - t0:>12.6f} {error:>14.2e} {iters:>8}")

    else:
        print(f"La matriz A para el valor N = {n} no fue creada por falta de memoria o un error.")
```

pedro

```
--- PASO 2: Resolución con Jacobi

       n   Tiempo (s)      Error máx    Iters
------------------------------------------
      10     0.000818       9.54e-07       19
      20     0.000183       9.54e-07       19
     100     0.000219       9.54e-07       19
     200     0.000348       9.54e-07       19
    1000     0.004606       9.54e-07       19
    2000     0.026054       9.54e-07       19
   10000     0.550733       9.54e-07       19
   20000     2.307695       9.54e-07       19
  100000   385.455227       9.54e-07       19
```

Problema 1. Enfoque Denso. Resolución Lineal.

Problema 1. Enfoque Denso.

```python
# --- PASO 3: Resolución mediante Sistemas Lineales.

print("\n\n--- PASO 3: Resolución con numpy.linalg.solve:\n")
print(f"{'n':>8} {'Tiempo (s)':>12} {'Error máx':>14}")
print("-" * 42)

for i, n in enumerate(N_VALORES):

    if matrices[i] is not None:

        A = matrices[i]
        b = np.ones(n, dtype=np.bool)

        try:
            t0 = time.perf_counter()
            x_sol = np.linalg.solve(A, b)
            t1 = time.perf_counter()

            x_exacta = np.full(n, 0.5)
            error = np.max(np.abs(x_sol - x_exacta))

            print(f"{n:>8} {t1 - t0:>12.6f} {error:>14.2e}")

        except (_ArrayMemoryError, ValueError, MemoryError) as e:
            print(f"ERROR: Sin memoria para n={n}: {e}")

    else:
        print(f"La matriz A para el valor N = {n} no fue creada por falta de memoria o un error.")
```

```
Resolución con numpy.linalg.solve
       n   Tiempo (s)      Error máx
------------------------------------------
      10     0.015089       1.11e-16
      20     0.000807       1.11e-16
     100     0.003798       1.11e-16
     200     0.009359       1.11e-16
    1000     0.029438       1.11e-16
    2000     0.105653       1.11e-16
   10000     4.182785       1.11e-16
   20000    26.949550       1.11e-16
ERROR: Sin memoria para n=100000: Unable to allocate 74.5 GiB for an array with shape (100000, 100000) and data type float64
```

caesar

Problema 1. Enfoque Denso. Tipos de datos cambiados.

Problema 1. Enfoque Denso.

```python
# ---------------------------------------- #
# --- FUNCIÓN: construcción de la matriz --- #
# ---------------------------------------- #

def construir_A(n):

    """
    Para construir esta matriz densa cíclica tridiagonal A de tamaño n x n, hemos seguido estos pasos:

    1. Rellenamos nuestra matriz A con ceros
    2. Usamos np.fill_diagonal para establecer la diagonal principal con el valor 4.
    3. Usamos la misma función pero en este caso para rellenar las diagonales adyacentes a la principal
    4. Añadimos las esquinas de forma manual.

    """

    # Creamos una matriz de tipo int8 ya que el valor mas alto dentro de esta va a ser 4.
    A = np.zeros((n, n), dtype=np.int8)

    # Diagonal principal.
    np.fill_diagonal(A, 4)

    # Rellenamos la subdiagona y la superdiagonal con valor -1.
    np.fill_diagonal(A[1:], -1)
    np.fill_diagonal(A[:, 1:], -1)

    # Rellenamos las esquinas con -1.
    A[0, n - 1] = -1
    A[n - 1, 0] = -1

    return A
```

caesar

Problema 1. Enfoque Denso. Tipos de datos cambiados.

Problema 1. Enfoque Denso.

```python
# --- PASO 1: Construcción de A y medición de tiempo/memoria para cada n.

print("\n\n--- PASO 1: Construcción de A y medición de tiempo/memoria\n")
print(f"{'n':>8} {'Tiempo (s)':>12} {'Memoria A ':>16}")
print("-" * 42)

matrices = [None] * len(N_VALORES)

# ITERAMOS TODOS LOS VALORES N. # Introducimos un contador para ir almacenando las matrices.
for i, n in enumerate(N_VALORES):

    # Estimamos el tamaño en memoria de la matriz. Cada valor ocupa un byte.
    memoria_estimada = (n**2) / (1024**2)

    # Indicamos el caso de estar pasandonos en memoria.
    if memoria_estimada > LIMITE_MEMORIA:
        print(
            f"{n:>8} {'—':>12} {memoria_estimada / 2**(10 ):>14.1f} GB (omitido: excede límite)"
        )
        continue

    # Iniciamos para cada matriz su contador de tiempo.
    t0 = time.perf_counter()
    A = construir_A(n)
    b = np.ones(n)
    t1 = time.perf_counter()

    # Guardamos la matriz en el array:
    matrices[i] = A

    # Medimos la memoria real una vez creada.
    memoria_real = A.nbytes / (1024**2)

    # En el caso de ocupar mas de 1GB cambiamos el formato de visualización.
    if(memoria_real < 2**(10)):
        print(f"{n:>8} {t1 - t0:>12.6f} {memoria_real:>14.4f} MB")

    else:
        print(f"{n:>8} {t1 - t0:>12.6f} {memoria_real / 2**10 :>14.4f} GB")
```

caesar

```
Construcción de A y medición de tiempo/memoria
       n   Tiempo (s)       Memoria A
------------------------------------------
      10     0.000048         0.0001 MB
      20     0.000027         0.0004 MB
     100     0.000029         0.0095 MB
     200     0.000044         0.0381 MB
    1000     0.000192         0.9537 MB
    2000     0.000476         3.8147 MB
   10000     0.009095        95.3674 MB
   20000     0.017717       381.4697 MB
   10000     0.009095        95.3674 MB
   20000     0.017717       381.4697 MB
  100000     0.098675         9.3132 GB
   10000     0.009095        95.3674 MB
   10000     0.009095        95.3674 MB
   20000     0.017717       381.4697 MB
   10000     0.009095        95.3674 MB
   10000     0.009095        95.3674 MB
   10000     0.009095        95.3674 MB
   20000     0.017717       381.4697 MB
   10000     0.009095        95.3674 MB
   10000     0.009095        95.3674 MB
   10000     0.009095        95.3674 MB
   20000     0.017717       381.4697 MB
  100000     0.098675         9.3132 GB
```

Problema 1. Enfoque Denso. ~~Tipos de datos cambiados~~

Problema 1. Enfoque Denso.

```python
# --- PASO 2: Resolución mediante Jacobi.

print("\n\n--- PASO 2: Resolución con Jacobi\n")
print(f"{'n':>8} {'Tiempo (s)':>12} {'Error máx':>14} {'Iters':>8}")
print("-" * 42)

for i, n in enumerate(N_VALORES):

    if matrices[i] is not None:

        A = matrices[i]

        # Los términos independientes son siempre uno.
        b = np.ones(n, dtype=np.bool)

        # Las soluciones se obtienen en base float32.
        x0 = np.zeros(n, dtype=np.float32)

        t0 = time.perf_counter()
        x_jac, iters = jacobi(A, b, x0, tol=TOL, maxit=MAXIT, verbose=False)
        t1 = time.perf_counter()

        # Calculamos el error sabiendo que la solución exacta es 0.5 para todos los casos.
        x_exacta = np.full(n, 0.5)
        error = np.max(np.abs(x_jac - x_exacta))

        print(f"{n:>8} {t1 - t0:>12.6f} {error:>14.2e} {iters:>8}")

    else:
        print(f"La matriz A para el valor N = {n} no fue creada por falta de memoria o un error.")
```

caesar

```
Resolución con Jacobi
       n   Tiempo (s)      Error máx    Iters
------------------------------------------
      10     0.002421       9.54e-07       19
      20     0.000221       9.54e-07       19
     100     0.000353       9.54e-07       19
     200     0.000918       9.54e-07       19
    1000     0.024099       9.54e-07       19
    2000     0.092139       9.54e-07       19
   10000     2.281619       9.54e-07       19
   20000     8.664568       9.54e-07       19
  100000   576.105229       9.54e-07       19
```

Problema 1. Enfoque Denso. Conclusiones.

El **método de Jacobi** es muy útil para resolver estas matrices y gana progresivamente en velocidad a la resolución lineal ya que tiene muchos valores nulos, por lo que **converge rápido**.

También, el método propio nos permite poder utilizar **tipos de datos menos precisos**, como float32 en vez de float64 para casos en los que queremos priorizar velocidad ante precisión de los cálculos.

Igualmente, en casos de matrices normales, el método factorización LU tiene una convergencia de 0(n^3), asegura resultados exactos y llega a funcionar más rápido.

Problema 2. Enfoque Cíclico. Importaciones y constantes.

```python
import time
import numpy as np
import taichi as ti

# Inicializar Taichi en gou nvidia (cuda)
ti.init(arch=ti.cuda, default_fp=ti.f64)

MAX_N = 2_000_000  # tamaño máximo soportado

# Campos en GPU (Taichi fields)
x_prev = ti.field(dtype=ti.f64, shape=MAX_N)  # iteración k-1
x_new = ti.field(dtype=ti.f64, shape=MAX_N)  # iteración k
diff_field = ti.field(dtype=ti.f64, shape=())  # acumulador escalar para diff
err_field = ti.field(dtype=ti.f64, shape=())  # acumulador escalar para error

# Constantes
N_VALORES = [
    10,
    20,
    100,
    200,
    1000,
    2000,
    10000,
    20000,
    100000,
    200000,
    1000000,
    2000000,
]
TOL = 1e-6
MAXIT = 10000
```

Problema 2. Enfoque Cíclico. Kernels Taichi.

```python
# Kernels
@ti.kernel
def init_cero(n: int):
    for i in range(n):
        x_prev[i] = 0.0


# En este kernel se calcula la nueva a partir de la previa. Además se paraleliza el bucle for
@ti.kernel
def jacobi_paso(n: int):
    for i in range(n):
        izq = x_prev[(i - 1 + n) % n]
        der = x_prev[(i + 1) % n]
        x_new[i] = (1.0 + izq + der) / 4.0


@ti.kernel
def copiar_new_a_prev(n: int):
    for i in range(n):
        x_prev[i] = x_new[i]


# Resultado escrito en diff_field[None] (campo escalar en gpu).
@ti.kernel
def calcular_diff(n: int):
    diff_field[None] = 0.0
    for i in range(n):
        d = ti.abs(x_new[i] - x_prev[i])
        ti.atomic_max(diff_field[None], d)


# El resultado exacto es x = 0.5 (viene en el encunciado), por lo que calculamos el error máximo respecto a esa solución.
@ti.kernel
def calcular_error(n: int):
    err_field[None] = 0.0
    for i in range(n):
        ti.atomic_max(err_field[None], ti.abs(x_prev[i] - 0.5))
```

(module) ti

Problema 2. Enfoque Cíclico. Función jacobi_ciclico

```python
def jacobi_ciclico(n, tol=1e-6, maxit=10000):
    if n > MAX_N:
        print(f"n={n} supera MAX_N={MAX_N}")
        return None, 0

    init_cero(n)                        # Inicializar x=0 en GPU

    for k in range(maxit):
        jacobi_paso(n)                  # Nuevo x_new en paralelo
        calcular_diff(n)                # Diff antes de actualizar
        diff = diff_field[None]



        if diff < tol:                  # Convergencia
            copiar_new_a_prev(n)
            calcular_error(n)
            return err_field[None], k + 1

        copiar_new_a_prev(n)            # x_prev <- x_new

    calcular_error(n)
    return err_field[None], maxit
```

Problema 2. Enfoque Cíclico. Compilación y benchmarks

```python
# Primera llamada compila los kernels en CUDA
print("\nCompilando kernels cuda")
jacobi_ciclico(10, tol=TOL, maxit=MAXIT)
print("Compilación completada.\n")

print(
    f"{'n':>10}  {'Tiempo (s)':>12}  {'Error máx':>12}  {'Iters':>8}  {'Mem. 2 vecs (KB)':>18}"
)
print("-" * 70)

for n in N_VALORES:
    if n > MAX_N:
        print(f"{n:>10}  —  (supera MAX_N={MAX_N:,})")
        continue

    # Sincronizar gpu antes de medir para evitar tiempos de compilación
    ti.sync()
    t0 = time.perf_counter()
    error, iters = jacobi_ciclico(n, tol=TOL, maxit=MAXIT)
    ti.sync()  # esperar a que la gpu termine antes de parar el reloj
    t1 = time.perf_counter()

    mem_kb = (2 * n * 8) / 1024.0 # Memoria: 2 vectores float64
    print(f"{n:>10}  {t1 - t0:>12.6f}  {error:>12.2e}  {iters:>8}  {mem_kb:>16.2f} KB")
```

```
[Taichi] version 1.7.4, llvm 15.0.1, commit b4b956fd, win, python 3.13.12
[Taichi] Starting on arch=cuda

Compilando kernels cuda
Compilación completada.

         n    Tiempo (s)     Error máx     Iters    Mem. 2 vecs (KB)
----------------------------------------------------------------------
        10      0.025234      9.54e-07        19              0.16 KB
        20      0.025071      9.54e-07        19              0.31 KB
       100      0.026804      9.54e-07        19              1.56 KB
       200      0.027955      9.54e-07        19              3.12 KB
      1000      0.023052      9.54e-07        19             15.62 KB
      2000      0.028320      9.54e-07        19             31.25 KB
     10000      0.022721      9.54e-07        19            156.25 KB
     20000      0.025783      9.54e-07        19            312.50 KB
    100000      0.026565      9.54e-07        19           1562.50 KB
    200000      0.026272      9.54e-07        19           3125.00 KB
   1000000      0.020090      9.54e-07        19          15625.00 KB
   2000000      0.034023      9.54e-07        19          31250.00 KB
```

Problema 2. Enfoque Cíclico. Conclusiones.

- <u>Memoria mínima: O(n)</u>

Solo se necesitan 2 vectores de tamaño n (x_prev y x_new), lo que permite manejar hasta n = 2.000.000 con apenas ~31 MB, frente a los GB que requeriría el enfoque denso.

- <u>Paralelización masiva en GPU</u>

El decorador @ti.kernel lanza el bucle for como un grid de hilos CUDA. Cada elemento se actualiza de forma completamente independiente (Jacobi puro), aprovechando al máximo la arquitectura paralela de la GPU.

Problema 2. Enfoque Cíclico. Conclusiones.

- <u>Tiempo casi constante para n pequeño</u>

El tiempo de ejecución se mantiene en ~0.02 segundos para todos los valores de n desde 10 hasta 2.000.000. Para n pequeño domina el overhead de sincronización; para n grande la paralelización masiva amortiza el coste, logrando una escalabilidad excelente.

- <u>Convergencia idéntica a CPU</u>

El algoritmo converge en 19 iteraciones con un error máximo de 9.54×10^{-7} (por debajo de la tolerancia 10^{-6}) para cualquier valor de n. Este comportamiento es idéntico al obtenido en CPU, al tratarse del mismo método de Jacobi.

Problema 2. Enfoque Cíclico. Conclusiones.

- <u>Conclusión global</u>

El método de Jacobi cíclico en GPU es óptimo para simulaciones en tiempo real como Stable Fluids: combina memoria mínima, paralelización masiva, tiempo constante y fácil integración con pipelines gráficos mediante Taichi/CUDA.

Problema 3. Enfoque Sparse. Función construir_A_sparse

```python
def construir_A_sparse(n):
    # Diagonales
    diag_principal = 4.0 * np.ones(n)
    diag_superior = -1.0 * np.ones(n - 1)
    diag_inferior = -1.0 * np.ones(n - 1)

    # Matriz tridiagonal
    A_sparse = sparse.diags(
        [diag_inferior, diag_principal, diag_superior],
        offsets=[-1, 0, 1],
        shape=(n, n),
        format="lil",
    )

    # Elementos cíclicos (esquinas)
    A_sparse[0, n - 1] = -1.0
    A_sparse[n - 1, 0] = -1.0

    # Convertir a CSC para spsolve
    return A_sparse.tocsc()
```

Estructura de la matriz sparse

Tipo: CSC (Compressed Sparse Column)
Elementos no nulos (NNZ): 3n

- Diagonales tridiagonales: 3n-2
- Elementos cíclicos: 2 elementos adicionales
- Total: 3n

Memoria: O(n) frente a O(n^2) denso

Solver: spsolve -> factorización LU esparsa (SuperLU)

Ventaja: Solo almacena los ~3n elementos no nulos, ignorando los ceros de la matriz nxn

Problema 3. Enfoque Sparse. Bucle de resolución.

```python
for n in N_VALORES:
    t0 = time.perf_counter()
    A_sp = construir_A_sparse(n)
    b = np.ones(n)
    x = spsolve(A_sp, b)
    t1 = time.perf_counter()

    elapsed = t1 - t0

    x_exacta = np.full(n, 0.5)
    error = np.max(np.abs(x - x_exacta))

    nnz = A_sp.nnz
    # Memoria aproximada del formato CSC: datos + índices de fila + punteros de columna
    mem_sparse_kb = (nnz * 8 + nnz * 4 + (n + 1) * 4) / 1024.0

    print(
        f"{n:>10}  {elapsed:>12.6f}  {error:>14.2e}  {nnz:>16}  {mem_sparse_kb:>16.2f} KB"
    )

    if elapsed > LIMITE_TIEMPO_S:
        print(f"  → Tiempo excesivo ({elapsed:.1f} s). Se detiene para n mayores.")
        break
```

n	Tiempo (s)	Error máx	NNZ (no nulos)	Mem. sparse (KB)
10	0.002393	5.55e-17	30	0.39 KB
20	0.000662	1.11e-16	60	0.79 KB
100	0.000780	1.11e-16	300	3.91 KB
200	0.000914	1.11e-16	600	7.82 KB
1000	0.002504	1.11e-16	3000	39.07 KB
2000	0.008296	1.11e-16	6000	78.13 KB
10000	0.037154	1.11e-16	30000	390.63 KB
20000	0.045826	1.11e-16	60000	781.25 KB
100000	0.242990	1.11e-16	300000	3906.25 KB
200000	0.463406	1.11e-16	600000	7812.50 KB
1000000	2.580104	1.11e-16	3000000	39062.50 KB

Problema 3. Enfoque Sparse. Conclusiones.

- <u>Memoria mínima: O(n)</u>

Al almacenar sólo los ~3n elementos no nulos, la memoria crece linealmente con n. Esto permite resolver sistemas de hasta n = 1000000 con apenas ~38MB (3.000.000 elementos x 12 bytes por elemento) frente a los 74 GB que requeriría la versión densa.

- <u>Precisión máquina</u>

El solver spsolve (SuperLU) realiza factorización LU esparsa y obtiene errores del orden de precisión doble (1.11e10-16), idénticos a los de numpy.linalg.solve pero con coste mucho menor.

Problema 3. Enfoque Sparse. Conclusiones.

- <u>Escalabilidad</u>

El tiempo de resolución crece de forma sublineal: desde 0.001s para n=10 hasta 2.374 s para n=1.000.000. La complejidad observada es aproximadamente O(n^1.1-1.2), muy por debajo del O(n^2) de Jacobi denso y del O(n^3) de la factorización LU densa.

- <u>Limitación: construcción lil -> csc</u>

La construcción en formato LIL y conversión a CSC añade un pequeño overhead para n grandes, pero sigue siendo negligible frente al tiempo de resolución (aproximadamente 0.1-0.2 s para n=1.000.000).

Conclusiones Finales. Comparativa de los tres enfoques.

Método	Memoria	Tiempo	Error máx	Escalabilidad
Denso (numpy)	~74 GB	Impracticable	-	O(n^2) en memoria
Jacobi Cíclico GPU	~31 MB	~0.022 s	9.54e-07	O(n) en memoria, tiempo ~constante
Sparse (scipy)	~38MB	~2.37 s	1.11e-16	O(n) en memoria, O(n^1.1) en tiempo